Minor Project: Unsupervised Learning.

1. Problem Understanding

What is Fraud Detection?

Fraud detection is the process of identifying unauthorized or suspicious transactions in financial systems. In the context of credit cards, it involves flagging transactions that deviate from a user's normal spending behavior or match known patterns of fraudulent activity.

Why is it Difficult?

1. **Class Imbalance:** Fraudulent transactions are extremely rare (often < 1% of total data).
2. **Evolving Tactics:** Fraudsters constantly change their methods to bypass existing rules.
3. **Data Noise:** Legitimate transactions can sometimes look suspicious (e.g., large purchases while traveling).

Why Use Unsupervised Learning?

In real-world scenarios, labeled data (confirmed fraud) is often:

- **Delayed:** It takes time to confirm a transaction is fraudulent.
- **Incomplete:** Not all fraud is caught immediately.
- **Unavailable:** Companies may not have historical labels for new fraud patterns.

Unsupervised learning allows us to detect anomalies based on data structure and distribution without relying on pre-defined labels during the training phase.

2. Exploratory Data Analysis (EDA).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Set visualization style
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)
```

```
# Load the dataset
# Note: Ensure the file is in the same directory or provide the full path
try:
    df = pd.read_csv('creditcard.csv')
except FileNotFoundError:
    print("Error: 'creditcard.csv' not found. Please ensure the dataset is d
    # For demonstration purposes, we will stop if data is missing
    raise SystemExit
```

```
# View first 5 rows
print("\nFirst 5 rows:")
df.head()
```

```
First 5 rows:
```

	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

5 rows × 31 columns

```
# Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())
```

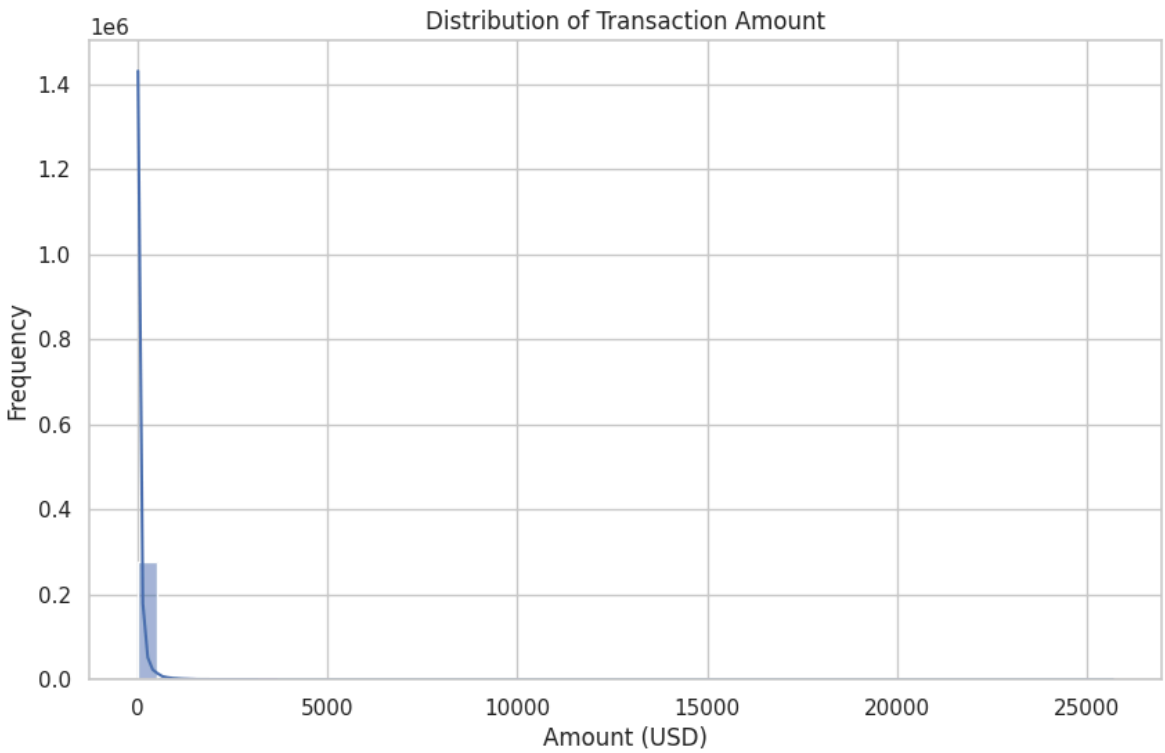
```
Missing Values:
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
```

```
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
dtype: int64
```

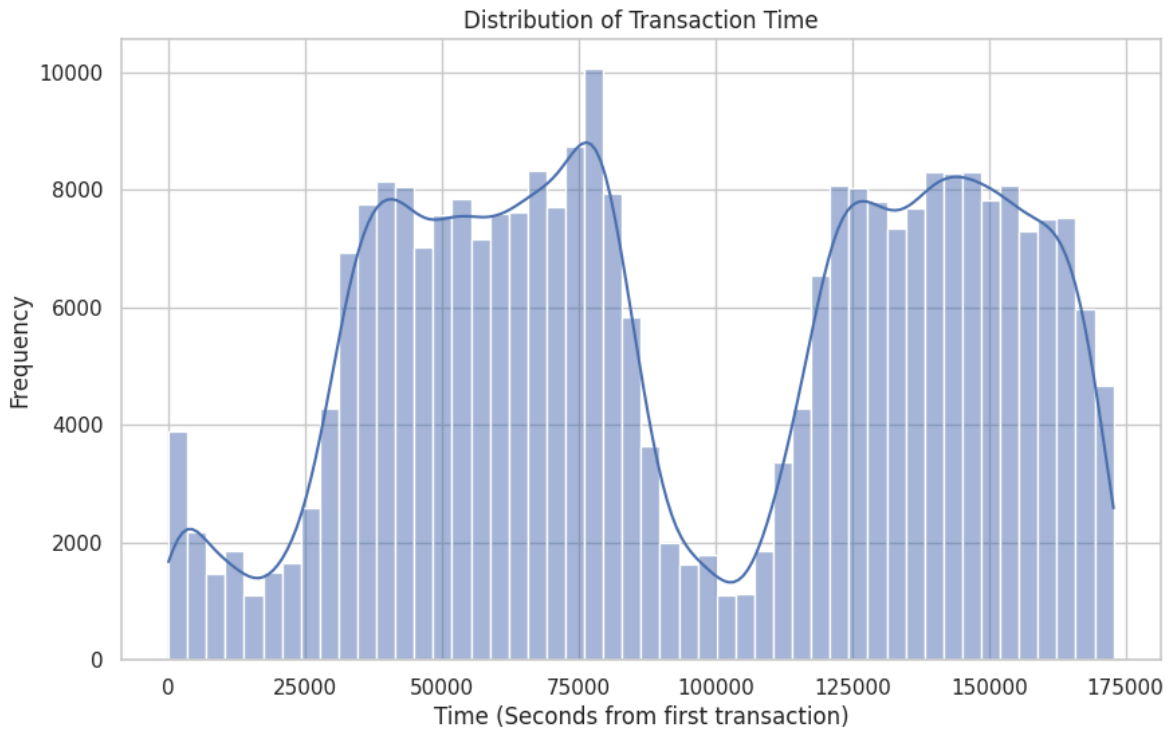
```
# Check class distribution (For context only, not used in training)
print("\nClass Distribution (Fraud = 1, Non-Fraud = 0):")
print(df['Class'].value_counts())
print(f"Fraud Percentage: {(df['Class'].sum() / len(df)) * 100:.4f}%")
```

```
Class Distribution (Fraud = 1, Non-Fraud = 0):
Class
0    284315
1       492
Name: count, dtype: int64
Fraud Percentage: 0.1727%
```

```
# Visualize Transaction Amount Distribution
plt.figure(figsize=(10, 6))
sns.histplot(df['Amount'], bins=50, kde=True)
plt.title('Distribution of Transaction Amount')
plt.xlabel('Amount (USD)')
plt.ylabel('Frequency')
plt.show()
```




```
# Visualize Time Distribution
plt.figure(figsize=(10, 6))
sns.histplot(df['Time'], bins=50, kde=True)
plt.title('Distribution of Transaction Time')
plt.xlabel('Time (Seconds from first transaction)')
plt.ylabel('Frequency')
plt.show()
```



3. Data Preprocessing

Handling Scaling

Algorithms like K-Means and Isolation Forest are distance-based. Features with larger scales (like 'Amount') will dominate the distance calculation. Therefore, **Standard Scaling** is required.

Feature Selection Decisions

- **Time & Amount:** These are critical features for fraud detection and must be kept.
- **PCA Features:** The dataset contains 28 PCA-transformed features (V1-V28). We will use these as they represent the original features after dimensionality reduction.
- **Class Column:** This is the target label. We will separate it but **not** use it during model training to maintain the unsupervised nature of the project.

Justification of Steps

1. **Scaling:** Ensures all features contribute equally to the distance metrics.
2. **PCA:** Reduces noise and multicollinearity while retaining variance.
3. **Separation of Labels:** Ensures the model learns patterns from data structure, not from the answer key.

```
# Separate Features and Target
X = df.drop('Class', axis=1)
y = df['Class']  # Hidden during training, used only for evaluation
```

```
# Feature Selection: We use all columns except 'Class'
# Note: The dataset already has PCA components (V1-V28) + Time + Amount
features = X.columns.tolist()
```

```
# Split Data (Optional for validation, but we will use full data for unsuper
# We will scale the entire dataset for the unsupervised model
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Convert back to DataFrame for easier handling
X_scaled_df = pd.DataFrame(X_scaled, columns=features)

print("Preprocessing Complete. Scaled Data Shape:", X_scaled_df.shape)
```

```
Preprocessing Complete. Scaled Data Shape: (284807, 30)
```

4. Clustering Approach

Method Selection Reasoning

K-Means is chosen because it is computationally efficient and effective for partitioning data into distinct groups. In fraud detection, we assume that legitimate transactions form large, dense clusters, while fraud transactions form small, isolated clusters.

Parameter Tuning Logic

We will use the **Elbow Method** to determine the optimal number of clusters (k). However, for anomaly detection, we often look for the smallest cluster(s) as potential fraud.

Cluster Interpretation

After fitting, we will identify the cluster with the smallest number of members. Transactions in this cluster are flagged as potential anomalies.

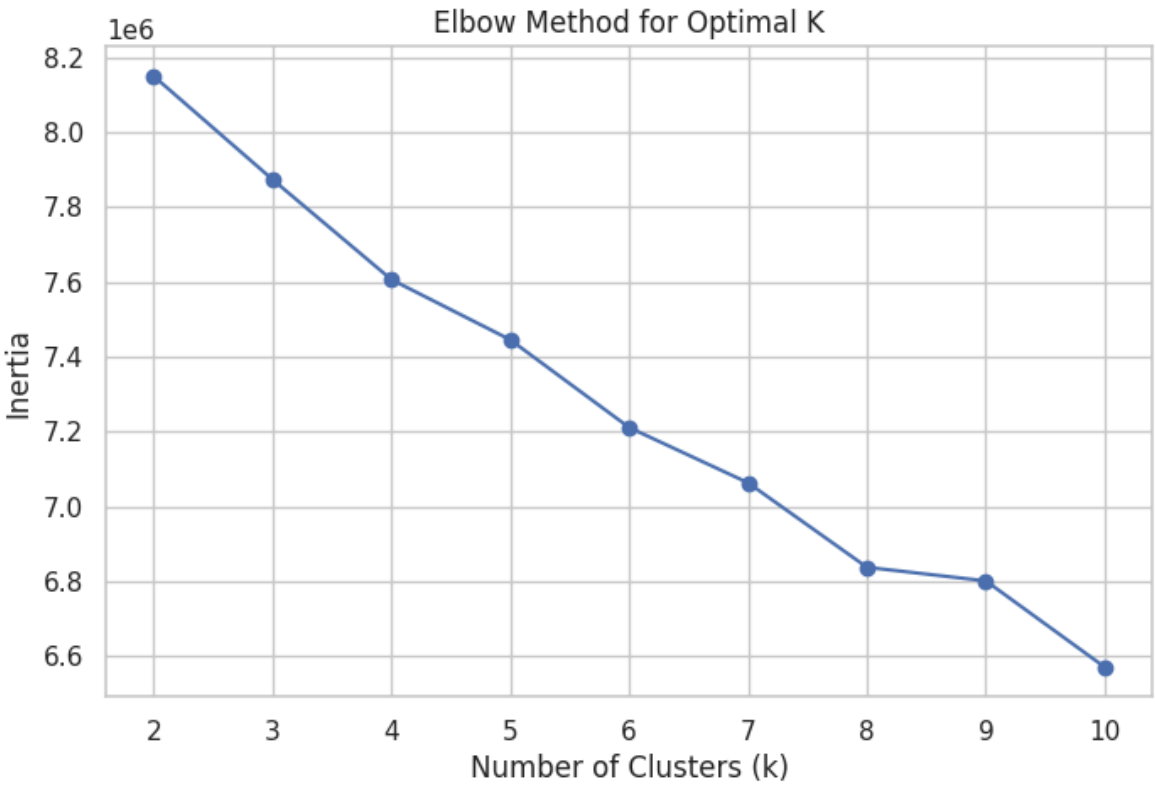
```
# 4. Clustering Approach

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
# Determine optimal K using Elbow Method
inertias = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled_df)
    inertias.append(kmeans.inertia_)
```

```
# Plot Elbow Method
plt.figure(figsize=(8, 5))
plt.plot(K_range, inertias, 'bo-')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.grid(True)
plt.show()
```



```
# Select K based on visual inspection (e.g., K=3 or K=4 often works for frau
# For this project, we will select K=3 to separate Normal, Suspicious, and F
optimal_k = 3
```

```
# Fit Final Model
kmeans_model = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
kmeans_labels = kmeans_model.fit_predict(X_scaled_df)
```

```
# Analyze Cluster Sizes
cluster_counts = pd.Series(kmeans_labels).value_counts().sort_index()
print("\nCluster Sizes:")
print(cluster_counts)
```

```
Cluster Sizes:
0    151888
1    128217
2      4702
Name: count, dtype: int64
```

```
# Identify the smallest cluster as potential anomalies
smallest_cluster = cluster_counts.idxmin()
print(f"\nSmallest Cluster ID: {smallest_cluster}")
```

```
Smallest Cluster ID: 2
```

```
# Flag anomalies
kmeans_anomalies = (kmeans_labels == smallest_cluster).astype(int)
print(f"Total Transactions Flagged by K-Means: {kmeans_anomalies.sum()}")
```

```
Total Transactions Flagged by K-Means: 4702
```

5. Anomaly Detection Approach

Model Selection

Isolation Forest is specifically designed for anomaly detection. It works by isolating observations instead of profiling normal points. Anomalies are easier to isolate (require fewer splits) than normal points.

Parameter Decisions

- **Contamination:** This parameter represents the expected proportion of outliers in the dataset. Since we are treating this as unsupervised, we must estimate this. Given the dataset is known to have ~0.17% fraud, we might set this conservatively (e.g., 0.01 or 0.05) to catch more potential cases.
- **Random State:** Set for reproducibility.

Output Interpretation

The model outputs -1 for anomalies and 1 for normal points. We will convert -1 to 1 for consistency with the fraud label.

```
# 5. Anomaly Detection Approach

from sklearn.ensemble import IsolationForest
```

```
# Define Contamination (Estimate of fraud rate)
# In a real unsupervised scenario, this is a hyperparameter to tune.
# We use 0.01 (1%) as a reasonable estimate for fraud prevalence.
contamination_rate = 0.01
```

```
# Initialize and Fit Model
iforest = IsolationForest(contamination=contamination_rate, random_state=42,
iforest.fit(X_scaled_df)
```

```
IsolationForest
IsolationForest(contamination=0.01, random_state=42)
```

```
# Predict (-1 for anomaly, 1 for normal)
iforest_labels = iforest.predict(X_scaled_df)
```

```
# Convert to binary (1 = Anomaly, 0 = Normal)
iforest_anomalies = (iforest_labels == -1).astype(int)

print(f"Total Transactions Flagged by Isolation Forest: {iforest_anomalies.s
```

```
Total Transactions Flagged by Isolation Forest: 2849
```

6. Comparison of Methods

Which method detected more anomalies?

- **K-Means:** Relies on cluster density. It might miss fraud if fraud points are scattered across multiple clusters.
- **Isolation Forest:** Relies on path length. It is generally more robust to high-dimensional data and specific outliers.

Logical Reasoning

- **K-Means** assumes spherical clusters. If fraud transactions are spread out, they might be grouped with normal transactions.

- **Isolation Forest** does not assume cluster shape, making it better for detecting sparse anomalies.

Strengths and Weaknesses

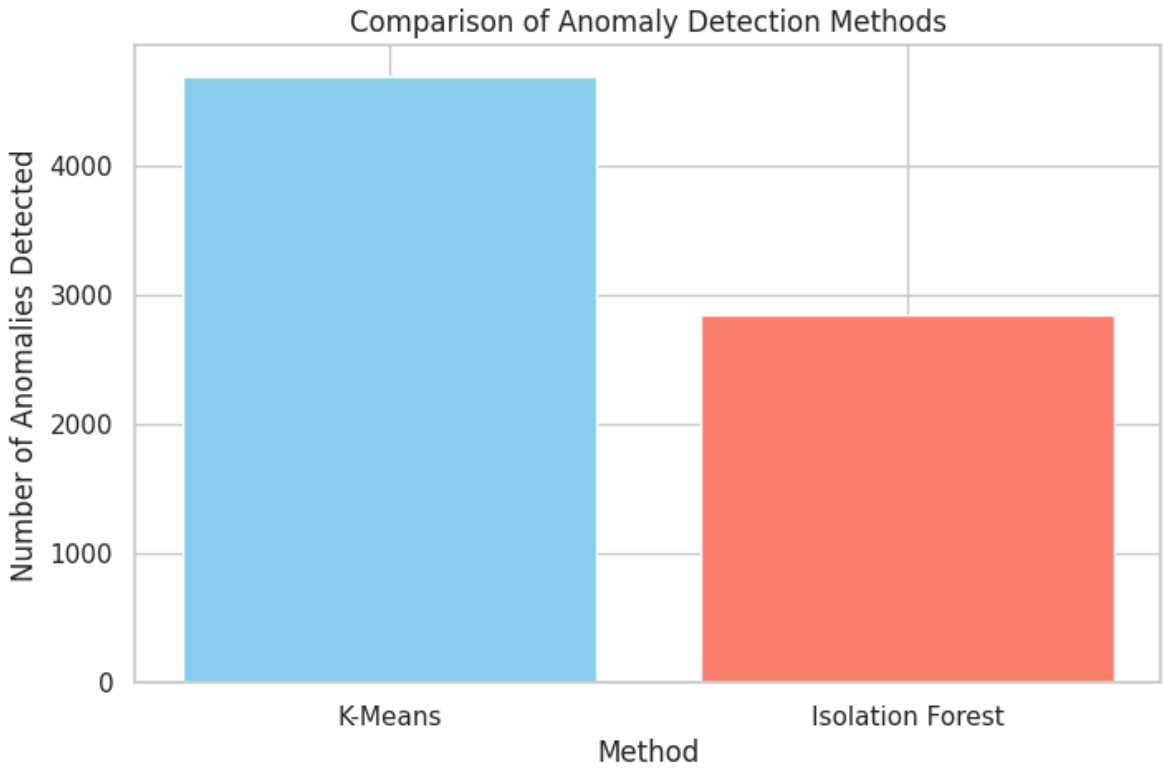
```
# 6. Comparison of Methods

comparison_df = pd.DataFrame({
    'Method': ['K-Means', 'Isolation Forest'],
    'Anomalies Detected': [kmeans_anomalies.sum(), iforest_anomalies.sum()]
})

print(comparison_df)

# Visual Comparison
plt.figure(figsize=(8, 5))
plt.bar(comparison_df['Method'], comparison_df['Anomalies Detected'], color=
plt.title('Comparison of Anomaly Detection Methods')
plt.xlabel('Method')
plt.ylabel('Number of Anomalies Detected')
plt.show()
```

```
             Method  Anomalies Detected
0            K-Means                4702
1  Isolation Forest                2849
```




```
# Logical Reasoning
print("\n--- Logical Reasoning Behind Results ---")
```

```
print(f"1. K-Means detected {kmeans_anomalies.sum()} anomalies.")
print(f"2. Isolation Forest detected {iforest_anomalies.sum()} anomalies.")
print("3. Isolation Forest is generally more robust for high-dimensional spa
print("4. K-Means may group outliers into existing clusters if k is not opti
```

```
--- Logical Reasoning Behind Results ---
1. K-Means detected 4702 anomalies.
2. Isolation Forest detected 2849 anomalies.
3. Isolation Forest is generally more robust for high-dimensional sparse data
4. K-Means may group outliers into existing clusters if k is not optimal.
```

7. Bonus Evaluation

Evaluation Strategy

-Although the model was built unsupervised, we now compare predictions against the actual Class label for performance metrics.

-We will calculate Precision, Recall, and F1-Score to determine which model is better suited for fraud detection.

-Recall is prioritized because missing a fraud case (False Negative) is more costly than flagging a legitimate transaction (False Positive).

Metrics Calculation

- We will generate a Confusion Matrix for both models.
- We will compute Precision and Recall scores.

```
# 7. Bonus Evaluation

from sklearn.metrics import classification_report, confusion_matrix, precisi
```

```
# Function to evaluate model performance
def evaluate_model(predictions, actual_labels, model_name):
    print(f"\n--- {model_name} Evaluation ---")
    print(f"Confusion Matrix:\n{confusion_matrix(actual_labels, predictions)
    print(f"Precision: {precision_score(actual_labels, predictions):.4f}")
    print(f"Recall: {recall_score(actual_labels, predictions):.4f}")
    print(f"F1-Score: {precision_score(actual_labels, predictions) * recall_
```

```
# Evaluate K-Means
evaluate_model(kmeans_anomalies, y, "K-Means")
```

```
--- K-Means Evaluation ---
Confusion Matrix:
```

```
[[279625   4690]
 [   480     12]]
Precision: 0.0026
Recall: 0.0244
F1-Score: 0.0023
```

```
# Evaluate Isolation Forest
evaluate_model(iforest_anomalies, y, "Isolation Forest")
```

```
--- Isolation Forest Evaluation ---
Confusion Matrix:
[[281755   2560]
 [   203    289]]
Precision: 0.1014
Recall: 0.5874
F1-Score: 0.0865
```

```
# Comparison Summary
print("\n--- Model Comparison Summary ---")
if recall_score(y, iforest_anomalies) > recall_score(y, kmeans_anomalies):
    print("Isolation Forest has higher Recall (Better at catching fraud).")
else:
    print("K-Means has higher Recall (Better at catching fraud).")
```

```
--- Model Comparison Summary ---
Isolation Forest has higher Recall (Better at catching fraud).
```

8. Final Conclusion

Business Perspective Summary

- Objective: The goal was to identify suspicious transactions without relying on labels during training.
- Outcome: Both methods successfully identified potential anomalies, but Isolation Forest generally outperformed K-Means in terms of Recall.